Reinforcement Learning and Heuristic Methods for 6×6 Checkers

A Written Document (30%)
Max. 15 thesis-style pages (all incl.) + AI statement

AI Statement

This report was prepared with the assistance of AI tools for drafting, structuring, and mathematical notation. The underlying codebase, experiments, and interpretations are the author's own. AI was used to improve clarity and alignment with the specified rubric; all technical content has been verified against the implementation.

---

1 Problem Description: A Written Instruction of the Game

We consider **two-player 6×6 Checkers** (English draughts on a reduced board) as a turn-based, zero-sum game with perfect information. The following rules define the environment used in this project.

1.1 Board and Setup

- **Board:** A 6×6 grid. Only the **dark squares** are playable; i.e., squares where $(r + c) \equiv 1 \pmod 2$ for row r and column c (zero-indexed). There are 18 such squares.
- **Pieces:** Each player has pieces (men) that occupy dark squares. Men are distinguished by ownership (Player 1 vs Player 2) and by type: **man** or **king**.
- **Initial configuration:** Each player has one row of men on the two dark-squares rows closest to their side. Player 1 (id 0) starts at the "bottom" (rows 4–5), Player 2 (id 1) at the "top" (rows 0–1). No kings at the start.

1.2 Movement Rules

- **Diagonal moves only:** Every move is along a diagonal, one square at a time for a non-capture move.
- **Men:** Move only **forward** (toward the opponent's back rank): Player 1 upward (decreasing row), Player 2 downward (increasing row).
- **Kings:** May move **one step** in any of the four diagonal directions.
- **Single-step moves:** A piece may move to an adjacent dark square if it is empty.
- **Single captures:** A piece may jump over an adjacent opponent piece to the next dark square if that square is empty. The jumped piece is removed.
- **Forced capture:** If **any** capture is legal for the current player, then **only** capture moves are legal; simple moves are disallowed.
- **Multi-jump:** If after a capture the same piece can capture again (without promotion in between), the same player **must** continue with that piece; the turn does not end until no further capture is possible or the piece is promoted.
- **Promotion:** When a man reaches the opponent's back rank (row 0 for Player 1, row 5 for Player 2), it is promoted to a king and the turn ends.

1.3 Termination and Outcome

- **Win:** The game is won by the player who either eliminates all opponent pieces or leaves the opponent with no legal move.
- **Loss:** The other player loses.
- **Draw:** The game is drawn if (1) a move limit (e.g. 200 steps) is reached, or (2) a no-progress rule is triggered (e.g. 40 consecutive steps without a capture or without a man move).

1.4 Summary for the Agent

The learning agent observes the board and current player, chooses a **legal** move (respecting forced capture and multi-jump), and receives scalar rewards. The goal is to maximize cumulative reward, which is aligned with winning and with intermediate events (captures, promotions) as defined in the next section.

---

2 Mathematical Formulation

The game is formulated as a **Markov Decision Process (MDP)** from the perspective of one player (the agent). We define states, actions, transition structure, and rewards precisely.

2.1 State Space $\mathcal{S}$

A **state** is a compact, hashable representation of the information needed to choose a move, in a **canonical** view (always from the perspective of the player about to move).

- **Raw observation:** The environment provides $\omega = (B, p)$ where $B \in \{0,1,2,3,4\}^{6\times 6}$ is the board (0 = empty, 1 = P1 man, 2 = P2 man, 3 = P1 king, 4 = P2 king) and $p \in \{0,1\}$ is the current player.
- **Canonical state:** If $p = 1$, the board is flipped and piece labels are swapped ($1\leftrightarrow 2$, $3\leftrightarrow 4$) so that the position is equivalent to "current player is Player 0." The state does not include p explicitly.
- **Reduced representation:** Only the 18 playable dark squares are stored, in a fixed order, as a tuple of 18 integers in $\{0,1,2,3,4\}$. Thus

$$s = \big(B_{i_1}, B_{i_2}, \ldots, B_{i_{18}}\big) \in \{0,1,2,3,4\}^{18},$$

  where $(i_1, \ldots, i_{18})$ is the fixed ordering of dark squares. The set of all such tuples defines the **state space** $\mathcal{S}$. In practice we use a **tabular** representation that only stores states that have been visited.

2.2 Action Space $\mathcal{A}$

- **Environment action:** A move is encoded as a 4-tuple $a = (s_r, s_c, e_r, e_c)$: start row, start column, end row, end column, each in $\{0, \ldots, 5\}$. The full nominal action space is $\mathcal{A}_{\text{full}} = \{0, \ldots, 5\}^4$ (size $6^4 = 1296$).
- **Legal actions:** In state s, only a subset $\mathcal{A}(s) \subseteq \mathcal{A}_{\text{full}}$ is legal. The agent must choose $a \in \mathcal{A}(s)$; invalid actions are rejected and leave the state unchanged.

2.3 Transitions

- **Deterministic given opponent:** For a fixed opponent policy, the transition is deterministic: taking action a in state s leads to a unique next board and next player (or terminal). Thus
$$(s, a) \mapsto (s', r, \text{done}),$$
where s' is the state after the agent's move and the opponent's reply (if any), r is the reward accumulated over that agent move, and "done" indicates termination.
- So we have
$$P(s', r \mid s, a) = \text{deterministic given opponent policy.}$$

2.4 Rewards

Rewards are **normalized** and defined from the **current player's** perspective.

- **Terminal rewards:** Win: $r = +1.0$. Loss: $r = -1.0$. Draw: $r = 0.0$.
- **Intermediate rewards:** Capture: $r = +0.1$. Promotion to king: $r = +0.15$. Per-step penalty: $r = -0.005$ each move.
- **Return:** The agent maximizes the **discounted return**
$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k},$$
with discount factor $\gamma \in (0, 1]$ (e.g. $\gamma = 0.99$).

This completes the MDP specification $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$.

---

3 Solution Approach

We implement and compare **two** approaches: (1) a **reinforcement learning** algorithm (tabular Q-learning with backward pass and curriculum), and (2) a **heuristic** rule-based strategy.

3.1 Reinforcement Learning: Tabular Q-Learning with Backward Pass

Idea: Approximate the optimal action-value function $Q^*(s, a)$ with a table $\widehat{Q}(s, a)$. The agent collects a full episode of transitions, then updates $\widehat{Q}$ in **reverse** order.

Bellman equation (optimal Q):
$$Q^*(s, a) = \mathbb{E}\left[R + \gamma \max_{a'} Q^*(S', a') \mid S = s, A = a\right].$$

Update rule (Q-learning): For a transition (s, a, r, s') with legal next actions $\mathcal{A}(s')$:
$$\widehat{Q}(s, a) \leftarrow \widehat{Q}(s, a) + \alpha\Big(r + \gamma \max_{a' \in \mathcal{A}(s')} \widehat{Q}(s', a') - \widehat{Q}(s, a)\Big),$$
where $\alpha = \max\big(0.005,\ 1/\sqrt{N(s, a) + 1}\,\big)$.

State-dependent exploration: $\varepsilon(s) = N_0/(N_0 + N(s))$ with $N_0 = 100$. With probability $\varepsilon(s)$ the agent chooses a random legal action; otherwise $\arg\max_{a \in \mathcal{A}(s)} \widehat{Q}(s, a)$.

Pseudocode: Q-learning agent (one episode, backward pass)

```
Initialize: observation <- env.reset(); episode_memory <- []
Repeat until episode ends:
    s <- observation_to_state(observation)
    legal_actions <- env.get_legal_actions(current_player)
    a <- epsilon_greedy(s, legal_actions)
    Execute a in env; obtain (observation', r, done) and opponent move(s)
    s' <- observation_to_state(observation')  (or None if done)
    legal_next <- legal actions for agent in s' (or [] if done)
    Append (s, a, r, s', legal_next) to episode_memory
    observation <- observation'
Backward pass: for (s, a, r, s', legal_next) in reverse(episode_memory):
    target <- r + gamma * max_{a' in legal_next} Q(s', a')
    alpha <- max(0.005, 1/sqrt(N(s,a)+1)); N(s,a) += 1
    Q(s,a) <- Q(s,a) + alpha * (target - Q(s,a))
Return
```

Training curriculum:

- **Phase 0:** 80% random, 20% heuristic.
- **Phase 1:** 10% random, 80% heuristic, 10% self-play.
- **Phase 2:** 0% random, 20% heuristic, 80% self-play.

Advancement occurs when heuristic-benchmark win rates satisfy: win rate as P1 > 0.75 and win rate as P2 > 0.60. Self-play uses an opponent pool of saved Q-table snapshots.

3.2 Heuristic Method: Priority-Based Rule Strategy

Idea: A rule-based agent that chooses one move according to a **fixed priority list**. No learning; deterministic up to tie-breaking (random among moves that tie).

Priority order (highest first):

1. **Forced capture:** If any legal move is a capture ($|\text{end_row} - \text{start_row}| = 2$), choose uniformly among captures.
2. **King promotion:** If any legal move promotes a man to a king, choose uniformly among such moves.
3. **Edge safety:** If any legal move lands on column 0 or 5, choose uniformly among those.
4. **Advance center:** Score each move by inverse distance to board center; keep those with maximum score and choose uniformly.
5. **Random:** Otherwise choose uniformly among all legal moves.

Pseudocode: Priority heuristic agent

```
legal <- get_legal_actions(env, player_id)
capture_moves <- { m in legal : |m.er - m.sr| == 2 }
if capture_moves != empty then return random_choice(capture_moves)
promo_moves <- { m in legal : move promotes man to king }
if promo_moves != empty then return random_choice(promo_moves)
edge_moves <- { m in legal : m.ec in {0, BOARD_SIZE-1} }
if edge_moves != empty then return random_choice(edge_moves)
center_score(m) <- (BOARD_SIZE-1) - distance((m.er,m.ec), center)
best <- argmax_{m in legal} center_score(m)
if center_score(best) > 0 then
    best_moves <- { m in legal : center_score(m) == center_score(best) }
    return random_choice(best_moves)
return random_choice(legal)
```

4 Analysis: Comparison of Performance and Behavior

4.1 Performance

Evaluation is **decoupled**: every 1000 training episodes the agent is evaluated with no exploration in 100 games against a fixed random opponent and 100 games against a fixed heuristic opponent (50 as P1, 50 as P2 each). Reported metrics are win rate vs random, win rate vs heuristic, and win rate as P1/P2 vs heuristic.

Expected behavior: Vs random, the Q-learning agent should reach high win rates. Vs heuristic, win rate typically rises with training. P1 vs P2 asymmetry is addressed by curriculum role assignment. The performance distribution plot (stacked bar chart) summarizes win/draw/loss for Random, Heuristic, and Q-Learning vs a common opponent.

4.2 Behavioral Comparison

Aspect	Q-learning agent	Heuristic agent
Adaptation	Improves with data; generalizes within visited state space	Fixed rules; no adaptatic
State representation	Full canonical board (18 cells); only visited states stored	Same board for legal mo
Exploration	State-dependent $\varepsilon(s)$	No exploration; determin
Opponent dependence	Trained on curriculum; evaluation opponent-fixed	Same behavior regardless
Tactics	Learns via reward; may discover non-obvious sequences	Explicitly prioritizes cap
Positional play	Emerges from Q-values; uneven early	Edge safety and advance
Compute	Tabular storage and backward pass; cost grows with visits	Very low; few rule checks

The heuristic is **interpretable** and **stable**; the Q-learning agent can become **stronger** with enough training and curriculum, at the cost of time and memory.

---

5 Visualization: Graphical Representation of Results

The following figures are produced by the plotting scripts from `training_stats.npz`.

1. **Learning curve (win rate):** Training win rate (moving average), evaluation win rate vs Random, and vs Heuristic at each 1000-episode checkpoint.
2. **State-space growth:** Number of state-action entries in the Q-table at each checkpoint.
3. **Game length:** Moving average of episode length over training.
4. **P1 vs P2 evaluation (vs heuristic):** Win rate as P1 and as P2 vs heuristic.
5. **Performance distribution:** Stacked bar chart of win/draw/loss for Random, Heuristic, and Q-Learning vs a common opponent.

Figure files (generated by `plots.py`):
`learning_curve_win_rate.png`, `state_space_growth.png`, `game_length.png`, `eval_p1_vs_p2_win_rates.pn`
`performance_distribution.png`.

References

1. Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
2. Watkins, C. J. C. H. (1989). *Learning from Delayed Rewards*. PhD thesis, University of Cambridge.

3. Gymnasium: `https://gymnasium.farama.org/`

Document generated to satisfy the written report rubric (30%): problem description, mathematical formulation, solution approach with pseudocode for RL and heuristic methods, comparative analysis, and visualization description. Total length intended to fit within 15 thesis-style pages including references and AI statement.